

TDTPSP_2

RDF, RDFS

A. TP Objective

The objective of this practical session is to understand and practice the fundamental concepts of RDF (Resource Description Framework) and RDFS (RDF Schema).

In TP1, you used XML to represent and validate structured academic information. In this TP, you will move from a hierarchical representation of data toward a graph-based representation of knowledge.

Important. This TP is primarily a learning activity about RDF and RDFS. It is not an implementation phase of the S2A project. The academic examples provide a concrete context that will be useful later in the semester.

Important. The S2A ontology is not being designed yet. The prefix `s2a:` used in this TP represents a temporary vocabulary created only for the exercises. It is not the final S2A ontology. For this TP, we will use:

```
@prefix s2a: <http://example.org/s2a/> .  
@prefix uni: <http://example.org/university/> .
```

The actual S2A ontology and vocabulary will be designed progressively in later practical sessions.

B. Exercises

Exercise 1. From Academic XML to RDF

In TP1, academic information was represented using XML. We now want to represent some of the same information using RDF.

Consider the following academic resources:

- [DSA201](#) - Data Structures and Algorithms
- [WEB301](#) - Web Technologies
- [SWKM701](#) - Semantic Web and Knowledge Management
- [DrSmith](#) - Professor Smith
- [Alice](#) - a student
- [Bob](#) - a student

Use the following namespaces:

```
@prefix s2a: <http://example.org/s2a/> .  
@prefix uni: <http://example.org/university/> .  
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
```

Represent the following facts in Turtle:

1. `DSA201` is a course
2. `WEB301` is a course
3. `SWKM701` is a course
4. `DrSmith` is a professor
5. `Dr Smith` teaches `DSA201`
6. `DSA201` has 4 credits
7. `DSA201` has the title "Data Structures and Algorithms"
8. `Alice` is a student
9. `Bob` is a student
10. `Alice` has passed `DSA201`
11. `Bob` is enrolled in `DSA201`
12. `SWKM701` has `DSA201` as a prerequisite
13. `SWKM701` has `WEB301` as a prerequisite

For example:

```
uni:DSA201 rdf:type s2a:Course .
uni:DrSmith rdf:type s2a:Professor .
uni:DrSmith s2a:teaches uni:DSA201 .
uni:DSA201 s2a:hasCreditValue 4 .
uni:DSA201 s2a:hasTitle "Data Structures and Algorithms" .
```

Questions:

- a. Identify the subjects, predicates and objects of several triples.
- b. Which objects are resources and which are literal?
- c. Which RDF statements represent relationships between academic resources?
- d. Compare the representation of the prerequisite relationship in TP1's XML and in RDF.
- e. What is the advantage of representing the prerequisite explicitly as an RDF property?

Exercise 2. Understanding RDF as a Graph

Using the RDF data from Exercise 1, draw a small graph representing the following information:

```
Alice → hasPassedCourse → DSA201
Bob → isEnrolledIn → DSA201
DrSmith → teaches → DSA201
SWKM701 → hasPrerequisite → DSA201
SWKM701 → hasPrerequisite → WEB301
```

Questions:

- a. What are the nodes of the graph?
- b. What are the relationships?
- c. Which resources have several relationships?
- d. How does this graph differ conceptually from the XML tree created in TP1?
- e. Why can RDF naturally represent interconnected information?

Exercise 3. Defining an Academic Vocabulary with RDFS

RDF allows us to represent facts, but we also need a vocabulary for describing the types of resources and the meaning of relationships. Using RDFS, define the following academic classes:

AcademicProgram
UndergraduateProgram
GraduateProgram
BachelorOfScience
MasterOfScience
PhD

The hierarchy is:

```
AcademicProgram
├── UndergraduateProgram
│   └── BachelorOfScience
└── GraduateProgram
    ├── MasterOfScience
    └── PhD
```

Represent this hierarchy using Turtle and `rdfs:subClassOf`.

For example:

```
s2a:GraduateProgram
    rdfs:subClassOf s2a:AcademicProgram .
```

Questions:

- What is the difference between a class and an instance?
- What is the meaning of `rdfs:subClassOf`?
- Why is it useful to organize academic programs into a hierarchy?
- What information can be inferred from the class hierarchy?

Exercise 4. Modeling Prerequisites and Student Progress

We now introduce a simple academic advising scenario.

The course [SWKM701](#) (Semantic Web and Knowledge Management) has two prerequisites:

- [DSA201](#) - Data Structures and Algorithms
- [WEB301](#) - Web Technologies

Alice has already passed [DSA201](#) but has not yet passed [WEB301](#).

Represent the situation using RDF. You should define and use appropriate properties such as:

s2a:hasPrerequisite
s2a:hasPassedCourse

For example:

```
uni:SWKM701
  s2a:hasPrerequisite uni:DSA201 ;
  s2a:hasPrerequisite uni:WEB301 .
uni:Alice
  s2a:hasPassedCourse uni:DSA201 .
```

Questions:

- What facts about **Alice** can be directly obtained from the RDF graph?
- Can we conclude that **Alice** has satisfied all prerequisites of **SWKM701**?
- What information would be necessary to determine this automatically?
- Would RDF alone be sufficient to implement a complete academic advising system?
- What additional querying or reasoning mechanisms could be useful?

Important observation. The RDF graph represents the facts and relationships. An advising application can then use these facts, together with queries and reasoning rules, to produce useful information for the student. This distinction will become important later when we study SPARQL, OWL and Knowledge Graph applications.

Exercise 5. Defining Properties with RDFS

We now want to describe the meaning of some properties used in the S2A domain. Define the following properties:

```
s2a:teaches
s2a:isEnrolledIn
s2a:hasPassedCourse
s2a:hasPrerequisite
```

For each property, specify an appropriate:

- `rdfs:domain`
- `rdfs:range`

For example:

```
s2a:hasPrerequisite
  rdfs:domain s2a:Course ;
  rdfs:range s2a:Course .
```

Students should determine suitable domain and range definitions for the other properties.

Questions:

- What is the purpose of `rdfs:domain`?
- What is the purpose of `rdfs:range`?
- Why is specifying domain and range useful?
- Can domain and range definitions lead to inferred types?

Exercise 6. RDF Data Correction and Completion

Consider the following Turtle document:

```
@prefix s2a: <http://example.org/s2a/> .
@prefix uni: <http://example.org/university/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

uni:Math201 a s2a:Course .
s2a:teaches uni:DrJones uni:Math201 .
uni:Math201 s2a:hasTitle "Linear Algebra" ;
    s2a:hasCreditValue 3 .
uni:Student uni:Charlie .
uni:Charlie s2a:isEnrolledIn uni:Math201 .
```

6.1. Identify at least three problems in the RDF/Turtle data. For each problem:

- identify the problematic statement;
- explain why it is problematic;
- provide a corrected version.

6.2. Add triples stating that:

- **DrEvans** is a professor;
- **DrEvans** teaches Machine Learning;
- **Machine Learning** is a course;
- **Machine Learning** has a credit value of 4;
- **David** is a student;
- **David** is enrolled in **Machine Learning**.

Use the vocabulary defined in the previous exercises.

6.3. Explain briefly why RDF is particularly appropriate for representing this type of interconnected academic information.

Exercise 7. Designing a Small RDFS Vocabulary for Smart Advising

The previous exercises introduced individual academic concepts. Now design a small RDFS vocabulary that could support part of the S2A Smart Academic Advisor. Define the following classes:

```
s2a:Course
s2a:Student
s2a:Professor
s2a:AcademicProgram
s2a:ProgramRequirement
```

Then define the following properties:

```
s2a:teaches
s2a:isEnrolledIn
s2a:hasPassedCourse
s2a:hasPrerequisite
```

```
s2a:isRequirementFor  
s2a:isPartOfProgram
```

For each property, specify an appropriate domain and range.

For example:

```
s2a:isRequirementFor  
  rdfs:domain s2a:Course ;  
  rdfs:range s2a:AcademicProgram .
```

And:

```
s2a:isPartOfProgram  
  rdfs:domain s2a:Student ;  
  rdfs:range s2a:AcademicProgram .
```

Final conceptual question:

Consider:

```
uni:Alice  
  rdf:type s2a:Student ;  
  s2a:isPartOfProgram s2a:MasterOfScience ;  
  s2a:hasPassedCourse uni:DSA201 .
```

and:

```
uni:SWKM701  
  rdf:type s2a:Course ;  
  s2a:hasPrerequisite uni:DSA201 ;  
  s2a:isRequirementFor s2a:MasterOfScience .
```

7.1. Explain how an academic advisor could use these facts to provide information to Alice.

7.2. What additional information, queries or reasoning would be required before making a reliable course recommendation?

C. Delivery Policy

The TP follows also a *two-stage work and submission process*.

Stage 1. In-class work: 75 minutes

During the supervised TP session, your group works on the exercises. At the end of the 75 minutes, you must upload the version of your work that you have achieved during the session.

This is your Submission 1, In-Class Checkpoint. The checkpoint *does not have to be complete*. The purpose is to provide evidence of your group's progress during the supervised session and to allow the instructor to identify difficulties and provide appropriate feedback. *This work will be evaluated from 10.*

During the TP

Learn → Experiment → Implement → Ask questions → Save → Upload checkpoint (*before the end of the session*).

Stage 2. Completion at home

After the TP session, your group continues working on the TP. You should, for example:

- Complete unfinished exercises
- Correct errors in the RDF/Turtle code
- Test your RDF data
- Complete the RDFS vocabulary
- Verify domain and range definitions
- Review the inference questions
- Improve the explanations
- Ensure that the complete RDF/RDFS model is coherent
- Prepare the final version of the TP

This work will be evaluated from 10.

After the TP

Complete → Test → Correct → Improve → Upload final version (*before Friday midnight*).

Expected Submission Structure (for both stages)

Your group should maintain one project folder containing, as appropriate:

TP2/

```
|— EX1_RDF_data.ttl
|— EX2_RDF_graph.pdf
|— EX3_RDFS_hierarchy.ttl
|— EX4_advising_data.ttl
|— EX5_RDFS_properties.ttl
|— EX6_corrected_data.ttl
|— EX7_S2A_vocabulary.ttl
```

The exact structure may be adapted according to the instructor's instructions.